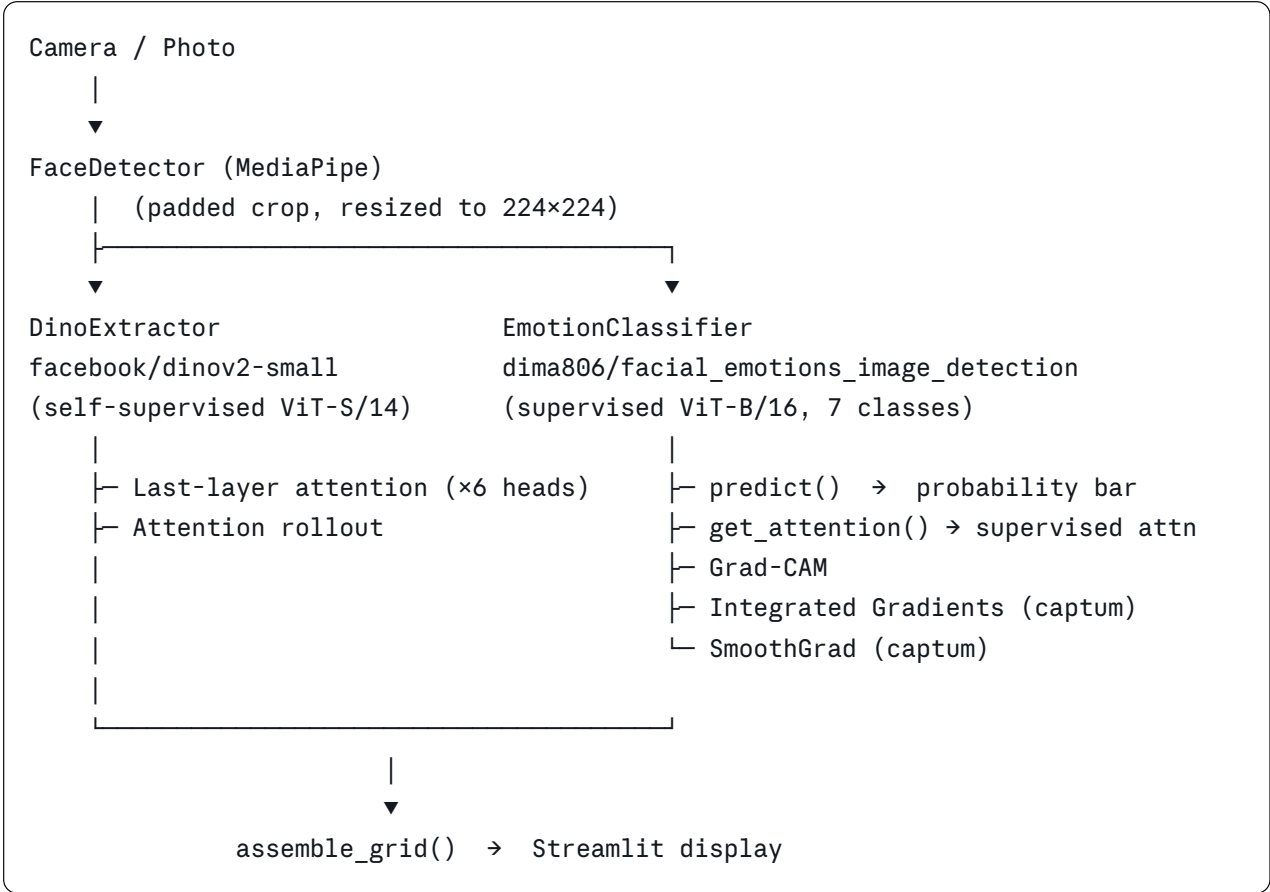


Architecture



Why two model paths?

The juxtaposition is the pedagogical core:

Panel	Source	What it teaches
DINO heads 1-4	DINOv2 (self-supervised)	What the model "sees" with no label supervision
Attention rollout	DINOv2	Propagated attention across all 12 blocks
Emotion ViT attn	Supervised ViT-B/16	How label-driven training changes attention
Grad-CAM	Supervised ViT	Which patches pushed the top-1 decision
Integrated Gradients	Supervised ViT	Axiomatic attribution satisfying completeness
SmoothGrad	Supervised ViT	Gradient stability under noise

Switching the target class in real time shows how DINO attention is *structurally stable* while gradient-based maps *shift* — a core intuition in modern XAI research.

Setup

Requirements

- Python 3.10+
- Webcam (built-in or USB)
- At least 8 GB RAM (models are loaded in fp32; reduce to fp16 on low-RAM machines)
- CUDA / Apple MPS optional but recommended for real-time stream mode

Install

```
bash

# 1. Clone / copy the project
cd xai_face_lab

# 2. Create virtual environment
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Run
streamlit run app.py
```

Models download automatically from Hugging Face on first run (~1-2 GB total). Subsequent runs load from cache (`~/ .cache/huggingface/`).

Usage

Snapshot mode (recommended for venues)

1. Open the app in a browser.
2. Select 📸 **Snapshot** in the sidebar.
3. Click **Take photo**.
4. All panels render within 2-5 seconds (CPU) or ~0.5 s (GPU).

Stream mode

1. Select 📺 **Stream**.
2. Click **Start** in the WebRTC widget.
3. Allow camera access in the browser.
4. XAI panels refresh every `PROCESS_EVERY_N_FRAMES` frames (configurable).

Venue note: Stream mode requires WebRTC-compatible network. If on a restricted conference WiFi, use Snapshot mode instead — it is equally impressive and more reliable.

Configuration

Edit `config.py` to tune:

Key	Default	Effect
EMOTION_MODEL_ID	dima806/...	Swap emotion model
IG_STEPS_DEFAULT	20	Integrated Gradients quality vs speed
SG_SAMPLES_DEFAULT	12	SmoothGrad quality vs speed
PROCESS_EVERY_N_FRAMES	4	Stream mode: XAI refresh rate
COLORMAP	inferno	Heatmap colormap
FACE_PADDING	0.30	Crop padding around detected face

XAI Method Reference

DINO Self-Attention

From the last transformer block of DINOv2-small. Each of the 6 attention heads attends to different facial regions. No label supervision: the model was trained purely by self-distillation. The emergent face segmentation is one of DINOv2's most striking properties.

Attention Rollout (Abnar & Zuidema 2020)

Recursively multiplies attention matrices across all 12 layers, adding residual connections to account for skip connections. Produces a single "where did information flow from" map.

Supervised ViT Attention

The emotion classifier's own last-layer attention, averaged over 12 heads. Contrast with DINO: supervised training sharpens attention toward emotionally diagnostic regions (mouth, eyes) at the cost of holistic coverage.

Grad-CAM (ViT adaptation)

Gradient of the target logit w.r.t. the last encoder block's patch token features, pooled over the hidden dimension. Class-conditional: changing the target class shifts the map.

Integrated Gradients (Sundararajan et al. 2017)

Integrates gradients along the straight-line path from a black baseline to the input image. Satisfies *completeness* (attributions sum to the output difference) and *sensitivity* axioms. More expensive than Grad-CAM but theoretically principled.

SmoothGrad (Smilkov et al. 2017)

Averages gradients over N copies of the input with added Gaussian noise. Reduces the "gradient shattering" artefacts common in plain saliency maps.

Extending

Swap in CelebA attribute classifier

```
python

# config.py
EMOTION_MODEL_ID = "your-hf-username/dinov2-celeba-probe"
```

Then train a linear probe on DINOv2 features:

```
bash

python train_probe.py --dataset celeba --epochs 10
```

(See `train_probe.py` for the training script template.)

Add TCAV

```
python

# xai_methods.py
from captum.concept import TCAV
# ...
```

TCAV computes a *directional derivative* in feature space, answering "does the model's representation encode concept X at all?" — a powerful complement to spatial saliency maps.

Citation

If you use this code in a publication or teaching material:

```
@misc{xai_face_lab_2025,
  title = {XAI Face Lab: Multi-method real-time explainability},
  author = {Panos, Brandon and I4DS, FHNW},
  year = {2025},
  url = {https://github.com/...}
}
```